

Design and Implementation of an Intelligent Trading Agent Using Machine Learning

Course: CEN352 Artificial Intelligence — Term Project

Group Members: Amel Sula, Aldinio Nurce, Dion Toska

Date: December 30, 2025

1. Introduction and Problem Formulation

Financial markets require sequential decision-making under uncertainty: prices change over time, and an agent must decide whether to buy, sell, or hold an asset without knowing future movements. The goal of this project is to design and implement an intelligent trading agent that operates on historical market data in an offline simulated environment and decides actions (Buy / Sell / Hold) using artificial intelligence methods taught in the course.

This project is educational and does not aim at real-world deployment. Instead, it demonstrates how AI techniques can be combined to build an agent that (1) learns patterns from data and (2) makes rational decisions based on learned predictions and explicit rules.

2. PEAS Framework

We define the task environment using the PEAS framework:

- Performance Measure (P):
 1. Prediction performance: Accuracy and F1-score for predicting next-day direction (Up/Down).
 2. Trading performance: total return (%), max drawdown (%), and number of trades.
- Environment (E): Historical daily stock data (OHLCV) used as an offline market simulation.

- Actuators (A): Three discrete actions: Buy, Sell, Hold (long-only).
- Sensors (S): Observed daily OHLCV values and engineered technical indicators (moving averages, RSI, volatility, momentum, volume change).

3. Agent Design

3.1 Agent Type

The implemented system is a learning agent with:

- a learning element (supervised ML classifier),
- a performance element (trading outcome in simulation),
- and a decision module that maps predictions to actions.

3.2 Task Environment Properties

- Partially observable: the agent cannot observe future prices.
- Stochastic: outcomes are uncertain due to market noise.
- Sequential: actions affect future portfolio state (cash/holdings).
- Dynamic (real-world) / fixed replay (offline): in real markets, dynamics change; in backtesting, we replay a fixed historical sequence.
- Discrete actions: Buy/Sell/Hold.
- Continuous state variables: prices and indicators are continuous.

3.3 Rationality

A rational agent selects actions that maximize expected performance according to its performance measure, given available information and constraints. Our agent follows a risk-aware policy: it trades only when conditions suggest an upward trend and the model's probability indicates an upward move, while also using risk controls (stop-loss and take-profit).

4. Methodology: Two AI Techniques

This project integrates two AI techniques as required.

4.1 Technique 1 — Statistical Learning (Supervised ML): Random Forest Classifier

We train a Random Forest classifier to predict next-day price direction.

Label (target):

For each day t , define

$$y_t = \begin{cases} 1 & \text{if } Close_{t+1} > Close_t \\ 0 & \text{otherwise} \end{cases}$$

Feature engineering: from OHLCV data we compute:

- daily return and log return,
- moving averages (SMA5, SMA10) and EMA10,
- rolling volatility (std of returns),
- momentum (difference over a short window),
- volume percent change,
- RSI (14).

Train/Validation/Test split (time-based):

To avoid look-ahead bias, we use a chronological split:

- Train: first 70%
- Validation: next 15%
- Test: last 15%

This respects the nature of time series data and ensures test results reflect generalization to unseen future periods.

Evaluation metrics:

We evaluate classification performance using:

- Accuracy
- F1-score (especially important when class performance differs)
- confusion matrix and classification report (precision/recall)

4.2 Technique 2 — Rule-Based Decision-Making (Trading Policy)

The ML model alone does not trade. We add a rule-based decision module that translates predictions into actions (Buy/Sell/Hold) with rational constraints.

Core idea: Use:

- Trend filter (moving averages) to avoid trading against the trend
- Model probability $P(\text{Up})$ as confirmation
- Risk controls (stop-loss/take-profit)

Policy summary (high-level):

- Buy when:
 - the market is in an uptrend (trend rules from moving averages), and
 - the model probability indicates Up with sufficient confidence.
- Sell when:
 - the trend breaks (downtrend), or
 - risk thresholds trigger exit (stop-loss or take-profit).
- Hold otherwise.

Trading constraints:

- long-only (no short selling),
- maximum holding of 1 share (simple, interpretable),
- transaction fee applied per trade (simulated friction).

This satisfies the “intelligent agent” requirement by combining learning-based perception with rule-based rational action selection.

5. Implementation

All code is implemented in Python and placed under the required /src folder.

Main modules:

- src/data.py: downloads data (optional) and loads CSV.
- src/features.py: cleans data, computes indicators, creates labels.
- src/train_model.py: trains Random Forest and reports validation metrics; saves artifact.

- `src/evaluate.py`: evaluates on the test split and prints final metrics.
- `src/agent.py`: implements the decision rules (Buy/Sell/Hold).
- `src/backtest.py`: simulates trading and compares to baselines; saves plot.

Saved outputs:

- trained model artifact: `models/AAPL_rf.joblib`
- portfolio comparison plot: `outputs/AAPL_portfolio_comparison.png`

6. Evaluation and Results

We report results for two evaluation layers:

1. Classifier performance (how well it predicts Up/Down)
2. Trading performance (how well the agent performs in the simulated environment)

6.1 Classifier Results (Test Set)

Test Metrics (final evaluation):

- Accuracy: 0.4867
- F1-score (Up class): 0.3077
- Confusion matrix $\begin{bmatrix} TN & FP \\ FN & TP \end{bmatrix}$:

$$\begin{bmatrix} 98 & 20 \\ 115 & 30 \end{bmatrix}$$

What the numbers mean:

- Accuracy 0.4867 means the model predicts the next-day direction correctly about 48.67% of the time on the test set.
- The confusion matrix indicates:
 - $TN = 98$: predicted Down and it was Down.
 - $FP = 20$: predicted Up but it was Down.
 - $FN = 115$: predicted Down but it was Up (missed Up days).
 - $TP = 30$: predicted Up and it was Up.

Interpretation:

- The model performs better at identifying Down days than Up days. In the classification report:
 - Class 0 (Down) has high recall (most down days were found).
 - Class 1 (Up) has precision = 0.60 but recall ≈ 0.2069 , meaning:
 - When the model predicts Up, it is correct fairly often (precision is decent),
 - but it misses many actual Up days (low recall).
- This conservativeness reduces the number of confident buy signals used by the trading agent.

6.2 Trading Results (Test Period)

We evaluate the trading agent on the test period and compare against two baselines.

Baselines:

1. Buy & Hold: buy 1 share at the start and hold until the end.
2. Random Trader: randomly chooses Buy/Sell/Hold daily (under the same constraints).

Backtest Outputs (Test Period):

- Agent: return = 0.04%, maxDD = 0.13%, trades = 6
- Buy & Hold: return = 0.52%, maxDD = 0.33%, trades = 1
- Random: return = 0.28%, maxDD = 0.31%, trades = 57

What these numbers mean:

- Return (%): percentage change in portfolio value from start to end of test period.
- Max drawdown (maxDD %): the largest peak-to-trough loss during the period; a risk measure.
- Trades: number of executed buy/sell actions (higher usually means higher fees and noise exposure).

Interpretation:

- Buy & Hold achieves the highest return in this test window. This is common in upward-trending periods because Buy & Hold captures the full move with minimal trading.
- The Random strategy made many trades (57). While it achieved 0.28% return in this window, random strategies are high-variance and not reliable; frequent trading also increases exposure to fees.
- Our Agent made only 6 trades and achieved:
 - low return (0.04%), but
 - significantly lower max drawdown (0.13%) than both baselines (0.33% and 0.31%).
- This demonstrates a risk-averse policy that limits exposure. The conservative trading behavior aligns with the classifier's low recall for Up movements: fewer detected Up opportunities means fewer entries, lower exposure, and lower drawdown.

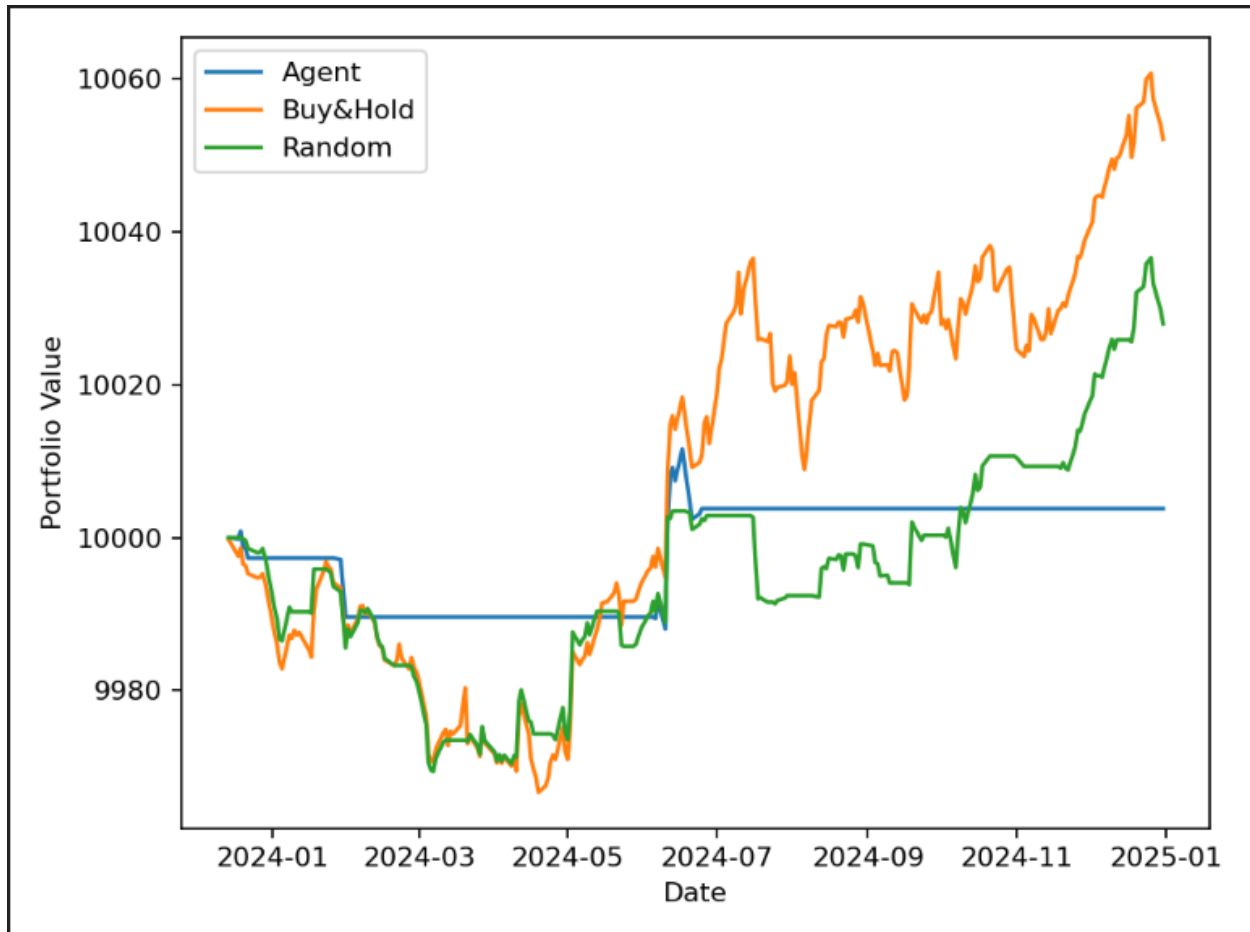


Figure 1: Portfolio value over time for Agent vs Buy & Hold vs Random

Insert the image: outputs/AAPL_portfolio_comparison.png

7. Ethical Considerations

Automated trading systems can contribute to market instability, amplify volatility, and provide unfair advantages to individuals or organizations with better access to AI technology and infrastructure. There is also a risk of misusing such systems without transparency, accountability, or proper controls.

This project is developed strictly for educational purposes. It operates on offline historical data, includes explicit evaluation and limitations, and does not claim financial usefulness. Responsible AI practice requires transparency about assumptions, avoiding misleading performance claims, and considering societal consequences if such systems are deployed at scale.

8. Future Work and Improvements

Several improvements could increase performance and realism:

1. Walk-forward validation and more robust time-series evaluation.
2. Better features (macro signals, additional indicators) and hyperparameter tuning.
3. Position sizing (not just 1 share) and improved portfolio/risk management.
4. Expand to multiple tickers and evaluate generalization across assets.
5. Reinforcement Learning for sequential decision-making (e.g., Q-learning) as an alternative decision policy.

References

- scikit-learn documentation (RandomForestClassifier, evaluation metrics)
- pandas documentation (time series operations and feature engineering)
- Yahoo Finance data source (via yfinance)